```
get '*a/foo/*b', to: 'test#index'
```
would match `zoo/woo/foo/bar/baz` with `params[:a]` equals `'zoo/woo'`, and `params[:b]` equals `'bar/baz'`.
By requesting `'/foo/bar.json'`, your `params[:pages]` will be equal to `'foo/bar'` with the request format of JSON. If you want the old 3.0.x behavior back, you could supply `format: false` like this:

```
get '*pages', to: 'pages#show', format: false
```
If you want to make the format segment mandatory, so it cannot be omitted, you can supply`format: true` like this:

```
get '*pages', to: 'pages#show', format: true
```

3.12 Redirection

You can redirect any path to another path using the `redirect` helper in your router:
```
get '/stories', to: redirect('/articles')
```
You can also reuse dynamic segments from the match in the path to redirect to:

```
get '/stories/:name', to: redirect('/articles/%{name}')
```
You can also provide a block to redirect, which receives the symbolized path parameters and the request object:

```
get '/stories/:name', to: redirect { |path_params, req|
"/articles/#{path_params[:name].pluralize}" }
get '/stories', to: redirect { |path_params, req|
"/articles/#{req.subdomain}" }
```
Please note that this redirection is a 301 "Moved Permanently" redirect. Keep in mind that some web browsers or proxy servers will cache this type of redirect, making the old page inaccessible.

In all of these cases, if you don't provide the leading host (`http://www.example.com`), Rails will take those details from the current request.

3.13 Routing to Rack Applications

Instead of a String like `'articles#index'`, which corresponds to the `index` action in the `ArticlesController`, you can specify any Rack application as the endpoint for a matcher:
```
match '/application.js', to: Sprockets, via: :all
```
As long as `Sprockets` responds to `call` and returns a `[status, headers, body]`, the router won't know the difference between the Rack application and an action. This is an appropriate use of `via: :all`, as you will want to allow your Rack application to handle all verbs as it considers appropriate.
For the curious, `'articles#index'` actually expands out to `ArticlesController.action(:index)`, which returns a valid Rack application.

3.14 Using `root`

You can specify what Rails should route `'/'` to with the `root` method:
```
root to: 'pages#main'
root 'pages#main' # shortcut for the above
```
You should put the `root` route at the top of the file, because it is the most popular route and should be matched first.
The `root` route only routes `GET` requests to the action.

You can also use root inside namespaces and scopes as well. For example:

```
namespace :admin do
```